

Week1

Before going through the content, I was initially confused about the difference between zero-knowledge proofs and encryption protocols, since both seem to involve hiding information. However, I now understand that they are different concepts. Encryption hides a message so that it can be decrypted and recovered later using the correct key. In contrast, zero-knowledge proofs you don't recover the hidden message but just prove whether the prover knows the secret or not.

This week, I read through PAZK Sections 1.1 to 2 and started reading Chapter 3. Chapter 3 was more technical and took more time to understand, especially the parts involving polynomial extensions and the Schwartz-Zippel Lemma, but I managed to understand the general intuition behind the concepts. I have summarized my understanding of the suggested questions below.

What are the properties of the proof system?

1. Completeness - if the statement is true, then a prover should be able to provide a convincing proof to the verifier
2. Soundness - if the statement is false, then no cheating prover should be able to convince the verifier
3. In the case of the arguments system, we assume that the soundness condition is only required to hold against prover strategies that run in polynomial time. In addition, the verification and proving procedures should ideally be efficient.

What are a "proof" and an "argument" respectively?

A proof is evidence that convinces the verifier that a statement is always true. It should satisfy both completeness and soundness.

An argument, is similar to a proof but its soundness only holds against a cheating prover that run in polynomial time.

What is a field, and what is a group?

So, a field is any set of elements with two main operations usually addition and multiplication that behave nicely, and the result stays within the same set. For example, real numbers form a field as any of these operations results in another real number except for division by zero.

Group is a set of number with one operation, such as addition or multiplication, that satisfies properties such as closure, associativity, identity, and inverses.

What is Reed-Solomon Encoding?

Reed-Solomon encoding represents the data as a low-degree polynomial and encodes it by evaluating the polynomial at many points. This produces a longer encoded version of the data so even if two original data differ at one position, their encoded version may differ in many positions. By evaluating the polynomial at a randomly chosen point, we can obtain a short fingerprint of the data. If two pieces of data are different, their corresponding polynomials

will usually give different values at a random point. Thus, it is possible to compare large data efficiently, with only a small probability of error.

What is Lagrange Interpolation?

In Reed-Solomon encoding, a vector can be interpreted as the coefficients of a univariate polynomial. However, the vector can also be interpreted as evaluations of a polynomial at fixed inputs, such as $0, 1, 2, \dots, n - 1$. So, Lagrange interpolation is a method for constructing a polynomial from its values at given points. So, for any n input-output pairs, there is a unique univariate polynomial of degree at most $n - 1$ that passes through those points. According to the theorem 2.4 in PAZK, this unique polynomial can be evaluated using $O(n)$ field operations, such as addition and multiplication.

What is the Schwartz-Zippel Lemma, and what is it used for?

My current understanding is that the Schwartz-Zippel Lemma says that if g is a nonzero low-degree polynomial of total degree at most d , then the probability that g evaluates to zero at a randomly chosen point from a finite set S is at most $\frac{d}{|S|}$. This means that given two different low-degree polynomials, they have a low probability to agree at a randomly chosen point. This allows a verifier to check claims about large data by testing them at random points rather than checking everything directly.

In addition to these questions, I was also wondering whether having one prover and multiple verifiers who trust each other would help in any way. I found that this is related to multi-verifier zero-knowledge (MVZK). Although I did not study it extensively, I gathered that multiple verifiers may be useful when verification is distributed, when the verifiers hold different information, or when no single verifier should be fully trusted.